

Lab 11 - AEAD

Intro

In this lab we explore **ChaCha20-Poly1305**, a modern **Authenticated Encryption with Associated Data (AEAD)** mode that provides both **confidentiality** and **integrity**. Unlike earlier stream cipher labs, where ChaCha20 only encrypted data, ChaCha20-Poly1305 also produces an **authentication tag** that verifies the data has not been altered. In addition to protecting the ciphertext, AEAD also authenticates selected, non-encrypted metadata called **Associated Authenticated Data (AAD)**. This means that even though the AAD remains visible in plaintext, any modification to it will cause decryption to fail. You will decrypt a message using an instructor-provided key, nonce, and ciphertext, then perform a series of controlled tampering experiments—changing a byte in the ciphertext, modifying the tag, and altering the AAD—to observe how the authentication mechanism detects and rejects unauthorized changes. By the end of the lab, you will understand how AEAD prevents both message tampering and metadata forgery, and why ChaCha20-Poly1305 is widely used in protocols such as TLS 1.3, SSH, and QUIC.

Part 01 - Encrypt

In Part 01, you will use ChaCha20-Poly1305 to encrypt a plaintext message and generate the three components required for decryption: the key, the nonce, and the ciphertext with authentication tag. ChaCha20 provides confidentiality by producing a keystream that is XORed with the plaintext, while Poly1305 produces a 128-bit authentication tag that binds the ciphertext and the Associated Authenticated Data (AAD). During this phase, you will also include a structured AAD value that is *not encrypted*, but will later be required to verify the integrity of the message. The output of Part 01 is a valid, authenticated ciphertext that can only be decrypted successfully if the correct key, nonce, and AAD are provided.

1. Copy the files down from repo
`scp -r repo@repo.homework.3310.com:/home/repo/lab11 .`
2. You should receive the following files:
 - a. part01
 - i. part01_02_chacha20poly1305_skel.py
 - ii. part01_plaintext.txt
 - iii. part01_nonce.bin
 - iv. part01_key.bin
 - v. part01_aad.bin (a file of size 0)

- b. part02
 - i. part02_ciphertext.bin
 - ii. part02_nonce.bin
 - iii. part02_key.bin
 - iv. part02_aad.bin (has a file size != 0)
 - c. part04
 - i. part04_aad_wrong.bin (has a file size != 0)
- 3. You will need to provide write permission to the directory that you downloaded.
- 4. You may also want to run dos2unix on `part01_02_chacha20poly1305_skel.py`
- 5. Modify `part01_02_chacha20poly1305_skel.py` so that you have aad data like the example in the skeleton code. DO NOT leave the aad as an empty binary string. Do not use my netid! Use yours, please.
- 6. Encrypt the message using the syntax: `python3 part01_02_chacha20poly1305_skel.py encrypt part01_key.bin part01_nonce.bin part01_plaintext.txt part01_ciphertext.bin part01_aad.bin`
- 7. Test your encryption (and that you actually got AAD data included) using the syntax: `python3 part01_02_chacha20poly1305_skel.py decrypt part01_key.bin part01_nonce.bin part01_ciphertext.bin part01_plaintext_generated.txt part01_aad.bin`
- 8. **Take a screenshot of your decryption capturing the AAD data being displayed.**
- 9. **Upload** your `part01_02_chacha20poly1305_skel.py` and your `part01_aad.bin` to Canvas.

Part 02 - Decrypt

You will decrypt the provided ciphertext using the key, nonce, and AAD. The critical concept here is that ChaCha20-Poly1305 will only decrypt the message if every component matches exactly. The ciphertext, tag, nonce, key, and AAD must all align with the originals used during encryption. If any value is incorrect or altered, the decryption algorithm will detect the mismatch and raise an authentication failure, preventing accidental or malicious tampering from producing misleading output. Successful decryption confirms that the ciphertext and its associated metadata are both authentic and intact.

- 10. Use `part01_02_chacha20poly1305_skel.py` to decrypt `part02_ciphertext`. You will be using `part02_aad.bin` which is a non-zero binary file.
- 11. **Take a screenshot of the decryption output including the AAD information.** It will be different from the part01 data.

12. Where is the authentication tag located? It is always 16 bytes appended to the very end of the ciphertext file. You can do the simple subtraction to see how long your message is
- a. `ls -al part02_plaintext_generated.txt`
 - b. Note the size of the file
 - c. Add 16 to that size
 - d. That should equal the size of the encrypted file
 - e. `ls -al part02_ciphertext.bin`
 - f. **Take a screenshot of both your file sizes to include in your lab report.**
13. You can also display the authentication tag
- a. `tail -c 16 part02_ciphertext.bin | xxd`
 - b. **Take a screenshot to include in your lab report.**

Part 03 - Flip one bit in the ciphertext

You will intentionally tamper with the ciphertext by flipping a single bit and attempting to decrypt again. Because the authentication tag is computed over the ciphertext, any alteration will cause a verification failure during decryption. This test demonstrates the integrity protection provided by AEAD. The system does not just encrypt data, it also ensures that the ciphertext cannot be modified without detection. When you perform this step, you should see an InvalidTag error, illustrating that altering encrypted data makes the entire message invalid.

14. Install rename to make life easier. You may have to update and upgrade first.
- ```
sudo apt install rename
```
15. Copy the ciphertext file from part02 into part03 directory. This way if you make a mistake you still have an original to return to. From inside the part02 directory type
- ```
cp part02_* ../part03/  
cd ../part03  
rename 's/part02/part03/g' *part02*
```
16. At the terminal type the following. It will flip one bit at offset 5.
- ```
python3 - << 'PY'
data = bytearray(open("part03_ciphertext.bin", "rb").read())
data[5] ^= 0x01
open("part03_ciphertext.bin", "wb").write(data)
PY
```
17. Now try to decrypt the altered ciphertext. It should give an error stating the message is invalid.
- ```
python3 ../part01/part01_02_chacha20poly1305_skel.py  
decrypt part03_key.bin part03_nonce.bin
```

```
part03_ciphertext.bin part03_plaintext_generated.txt
part03_aad.bin
```

18. **Take a screenshot of the output including the error message and include it in your lab report.**

Part 04 - Change the AAD

You will modify the Associated Authenticated Data (AAD) and attempt decryption again. The AAD is not encrypted and remains readable, but it is included in the authentication process. This means that changing even a single character in the AAD should cause decryption to fail, just like flipping a bit in the ciphertext. This behavior demonstrates that AEAD protects not just secret message contents, but also important metadata. This ensures both message integrity and authenticity. The failure here reinforces the idea that “unencrypted” does not mean “unprotected.”

19. Again, copy the ciphertext file from part02 into part04 directory. This way if you make a mistake you still have an original to return to. There will already be a file named part04_aad_wrong.bin in there. From inside the part02 directory type

```
cp part02_* ../part04/
cd ../part04
rename 's/part02/part04/g' *part02*
```

20. I only changed one value in part04_add_wrong.bin but it should not decrypt the message.

```
python3 ../part01/part01_02_chacha20poly1305_skel.py
decrypt part04_key.bin part04_nonce.bin
part04_ciphertext.bin part04_plaintext_generated.txt
part04_aad_wrong.bin
```

21. **Take a screenshot of the output with the error message displayed.**
22. **Explain the difference between confidentiality and integrity in your own words, using this lab as your example.** Specifically, think about the part of the ciphersuite that provides confidentiality and the portion that provides integrity.
23. AEAD gives confidentiality + integrity + authenticity. Historically, systems used Encrypt-then-MAC. **What benefits does an integrated AEAD design provide beyond convenience?**
24. **Compare AES-GCM to ChaCha20Poly1305. What are some similarities and differences?**

Lab 11 Template

Part 01:

1. **Take a screenshot of your decryption capturing the AAD data being displayed.**
(10 points)
2. **Upload your `part01_02_chacha20poly1305_skel.py` and your `part01_aad.bin` to Canvas.**
(10 points, 5 points each item)

Part 02:

3. **Take a screenshot of the decryption command and output including the AAD information.**
(10 points)
4. **Take a screenshot of both your file sizes to include in your lab report.**
(10 points, 5 points each item)
5. **Take a screenshot of your authentication tag (last 16 bytes) to include in your lab report.**
(10 points)

Part 03:

6. **Take a screenshot of the command and output including the error message for the flipped bit and include it in your lab report.**
(10 points)

Part 04:

7. **Take a screenshot of the command and output including the error message for the changed AAD information and include it in your lab report.**
(10 points)
8. **Explain the difference between confidentiality and integrity in your own words, using this lab as your example.**
(10 points, 5 points each confidentiality, 5 points integrity)
9. **What benefits does an integrated AEAD design provide beyond convenience?**
(10 points)
10. **Compare AES-GCM to ChaCha20Poly1305. What are some similarities and differences?**
(10 points)